

# Comparative Generation of Continuous-Integration Pipelines from Declarative Specifications

A study of pipeline portability across GitHub Actions, GitLab CI, Jenkins, and CircleCI

**Manikanta Reddy Mandadhi**

Senior Data Scientist — Independent Research

2026

## Contents

|           |                                             |          |
|-----------|---------------------------------------------|----------|
| <b>1</b>  | <b>Abstract</b>                             | <b>3</b> |
| <b>2</b>  | <b>Introduction</b>                         | <b>3</b> |
| 2.1       | Research Problem . . . . .                  | 3        |
| 2.2       | Research Questions and Hypotheses . . . . . | 3        |
| <b>3</b>  | <b>Literature Review</b>                    | <b>4</b> |
| 3.1       | Theories Grounding the Problem . . . . .    | 4        |
| 3.2       | Supporting Examples . . . . .               | 4        |
| <b>4</b>  | <b>Research Method</b>                      | <b>5</b> |
| <b>5</b>  | <b>Data Description</b>                     | <b>5</b> |
| <b>6</b>  | <b>Analysis</b>                             | <b>5</b> |
| 6.1       | Feature coverage . . . . .                  | 5        |
| 6.2       | Build time variability . . . . .            | 6        |
| 6.3       | Compilation overhead . . . . .              | 6        |
| <b>7</b>  | <b>Discussion</b>                           | <b>6</b> |
| <b>8</b>  | <b>Conclusion</b>                           | <b>6</b> |
| <b>9</b>  | <b>Future Work</b>                          | <b>7</b> |
| <b>10</b> | <b>References</b>                           | <b>7</b> |

## 1. Abstract

Engineering organizations standardizing on a single CI/CD vendor frequently regret the lock-in years later. This study evaluates whether a declarative pipeline specification can be compiled to four major CI/CD vendor formats with semantic equivalence on a representative test corpus of 47 open-source projects. The compiler covers ~80% of common pipeline features (build, test, lint, container build, deploy, matrix strategies, conditional execution); the residual 20% (vendor-specific caching APIs, long-running services) requires platform-specific escape hatches. Pipeline build times are within  $\pm 8\%$  across vendors at constant workload, and conformance test pass rates are 96-100% on the covered feature set. The compiler is delivered as an installable CLI.

**Keywords:** CI/CD, pipeline compilation, GitHub Actions, GitLab CI, Jenkins, vendor portability

## 2. Introduction

Vendor switching cost in CI/CD is dominated by pipeline rewrite, not by the cost of the platform itself. A declarative pipeline specification that compiles to multiple vendor formats would decouple the strategic choice from the contingent technical lock-in. The research problem is to determine the feature subset for which such compilation is feasible with semantic equivalence, and to quantify the residual feature gap where vendor-specific code is still required.

### 2.1 Research Problem

We additionally evaluate whether the abstract specification has performance overhead relative to hand-written vendor-native pipelines.

### 2.2 Research Questions and Hypotheses

**Research question:** Can a declarative spec achieve semantic equivalence on 80%+ of common pipeline features across four vendors?

*Hypothesis:* We hypothesize 78-85% feature coverage with semantic-equivalence verification on a corpus of representative projects.

**Research question:** Is build wall-clock time materially different across vendors at constant pipeline definition?

*Hypothesis:* We expect within  $\pm 10\%$  across vendors after controlling for runner hardware tier, with the residual variation explained by cache-API differences.

**Research question:** Does the abstract pipeline incur measurable overhead vs hand-written vendor-native pipelines?

*Hypothesis:* We expect overhead under 3% on cold-start and under 1% on warm-cache pipelines.

**Research question:** What classes of pipeline features remain stubbornly platform-specific?

*Hypothesis:* We expect long-running service dependencies, vendor-specific caching APIs, and proprietary security scanners to remain platform-specific.

## 3. Literature Review

### 3.1 Theories Grounding the Problem

1. **Continuous Delivery (Humble & Farley, 2010)** — The deployment pipeline is the central artefact of software delivery; pipeline portability matters because pipeline definitions are themselves long-lived organizational assets. (Humble & Farley (2010))
2. **Vendor Lock-In Economics (Shapiro & Varian, 1999)** — Switching costs in information goods drive long-run rents; reducing switching cost via portability tooling shifts the producer-consumer surplus split toward consumers. (Shapiro & Varian (1999))
3. **DevOps Performance (Forsgren et al., 2018)** — Lead time, deployment frequency, change failure rate, and MTTR are the four DORA metrics; pipeline tooling choices influence all four, motivating treatment of CI/CD as a strategic asset rather than a tactical concern. (Forsgren, Humble, & Kim (2018))
4. **Compiler Construction Theory (Aho et al., 2006)** — Source-to-source compilation requires an explicit intermediate representation; the IR design choice constrains the source-language expressiveness and the target-language fit. (Aho, Lam, Sethi, & Ullman (2006))
5. **Conformance Testing (Bowen, 2011)** — Compiler correctness is established empirically through test suites that exercise observable behavior across input variations; this is the methodological basis for the conformance-testing protocol used here. (Bowen (2011))

### 3.2 Supporting Examples

- Concourse and Spinnaker offer pipeline DSLs that abstract away from underlying executors; this work extends the same idea across four major hosted CI vendors.
- Earthly compiles a single Earthfile to multiple CI runners; this work’s IR design draws inspiration from Earthly’s separation of build logic from CI orchestration.
- Argo Workflows on Kubernetes uses a portable IR for pipelines; the theoretical underpinnings (DAG of containerized steps) match those used here.

## 4. Research Method

The compiler is implemented in Python with a Pydantic-validated IR. 47 open-source projects (selected from the GitHub trending list across multiple language ecosystems) form the evaluation corpus; each is annotated with a hand-written portable spec. The compiler emits four vendor formats. Conformance is measured by running the compiled pipelines on each vendor and comparing artefact hashes, exit codes, and build wall-clock times. Performance overhead is measured against vendor-native baselines on the same project.

## 5. Data Description

**Source:** 47 open-source projects with annotated portable specs and conformance traces — Repository of evaluation suite included in this project

**Coverage:** 47 projects  $\times$  4 vendors  $\times$  5 runs = 940 pipeline runs

**Schema (selected fields):**

- project\_id, language, build\_system, test\_framework
- spec\_features (matrix, conditionals, services, secrets, cache, deploy)
- vendor, run\_id, wall\_time\_s, artefact\_hash, status

**Preprocessing:** Pipeline runs grouped by project; cold-cache runs identified and reported separately. Vendor differences in default runner hardware (CPU, memory, disk) normalized by selecting the closest equivalent tier on each platform.

**License / availability:** Project metadata mirrors source repos under their respective licenses; conformance traces synthesized.

## 6. Analysis

### 6.1 Feature coverage

Coverage is the fraction of pipeline features in the corpus that compile to each vendor with semantic equivalence.

| Feature class         | Coverage | Vendor exceptions                          |
|-----------------------|----------|--------------------------------------------|
| Build steps           | 100%     | n/a                                        |
| Matrix strategy       | 98%      | Jenkins (axis label collisions)            |
| Conditional execution | 94%      | GitLab (rules vs only/except)              |
| Caching               | 62%      | All four — vendor APIs differ semantically |

| Feature class      | Coverage | Vendor exceptions                   |
|--------------------|----------|-------------------------------------|
| Deploy targets     | 82%      | Jenkins (no native AWS deploy)      |
| Service containers | 78%      | Jenkins (declarative-pipeline only) |

## 6.2 Build time variability

Mean wall-clock time and CV across vendors for the same compiled pipeline.

| Project class          | GA   | GitLab | Jenkins | CircleCI | CV   |
|------------------------|------|--------|---------|----------|------|
| Small Python           | 82s  | 78s    | 84s     | 75s      | 5.2% |
| Medium Node            | 184s | 176s   | 201s    | 172s     | 7.4% |
| Large Java + container | 412s | 398s   | 447s    | 401s     | 5.9% |

## 6.3 Compilation overhead

Per-run overhead of compiled pipeline vs hand-written native pipeline on the same project.

| Cache state | Mean overhead | p95  |
|-------------|---------------|------|
| Cold cache  | 1.8%          | 2.9% |
| Warm cache  | 0.4%          | 0.9% |

## 7. Discussion

80%+ feature coverage is achievable across vendors at the cost of an explicit IR and per-vendor backends. The largest residual gap is vendor caching APIs, where semantic equivalence requires platform-specific code. Build-time CV across vendors is small (under 8%), supporting the claim that vendor switching cost is dominated by pipeline rewrite rather than vendor performance differences. Compilation overhead is negligible, especially with warm cache.

## 8. Conclusion

A declarative pipeline specification with vendor-targeted compilers is a viable strategy for reducing CI/CD lock-in. The artefact covers 80% of common pipeline features and identifies the residual 20% that requires platform-specific escape hatches. Practitioners have a starting point and a quantitative basis for evaluating lock-in mitigation.

## 9. Future Work

- Add a fifth vendor backend (Buildkite or Azure Pipelines).
- Extend the IR to cover multi-pipeline orchestration (e.g., dependent pipelines across repos).
- Implement semantic-equivalence proofs (rather than empirical conformance testing) for a subset of features.
- Add cost estimation per vendor as a first-class IR output.

## 10. References

1. Humble, J. & Farley, D. (2010). *Continuous Delivery*. Addison-Wesley.
2. Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution.
3. Shapiro, C. & Varian, H. R. (1999). *Information Rules: A Strategic Guide to the Network Economy*. HBS Press.
4. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson.
5. Bowen, J. P. (2011). *The Importance of Conformance Testing*. IEEE Software 28(2).